



บทที่ 6 ตอนที่ 2

คุณลักษณะและเมธอดของคลาสและของอ็อบเจกต์

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรรยาโรจน์

การเขียนโปรแกรมเชิงวัตถุ

การห่อหุ้ม
Encapsulation

การสืบทอด
Inheritance

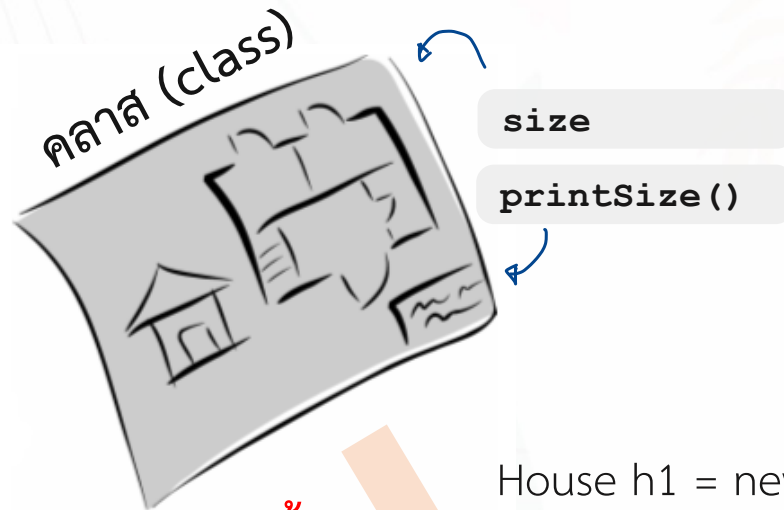
การมีได้
หลากหลายรูปแบบ
Polymorphism

คีย์เวิร์ด static

```
public class House{  
    public static int size ;  
    public int house_id ;  
    public static void printSize(){ ... }  
    public void printHouseID(){ ... }  
}
```

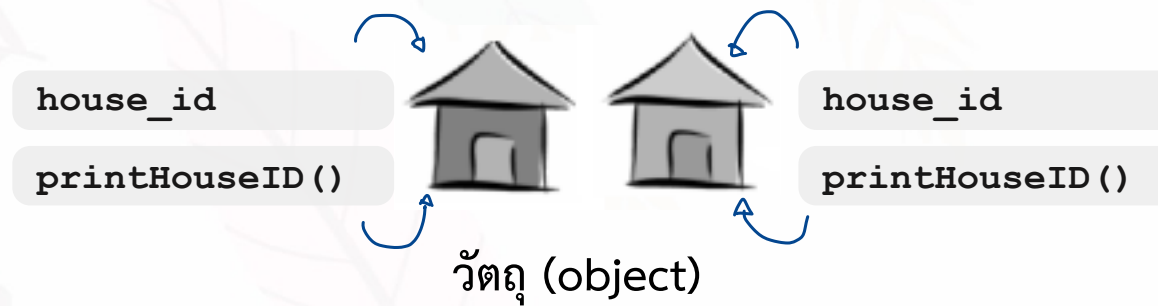
Attribute , Method ที่มี static เป็นของ class
ถ้าไม่มี static เป็นของ object

คีย์เวิร์ด static เป็นส่วนหนึ่งของ OOP และมีหน้าที่กำหนดให้ (1) attribute หรือ (2) method เป็นส่วนของคลาสแทนที่จะเป็นส่วนของอ็อบเจกต์ของคลาสนั้น ๆ นั่นหมายความว่า คุณสามารถเข้าถึงตัวคุณลักษณะหรือเมธอด โดยไม่ต้องสร้างอ็อบเจกต์จากคลาสนั้นก่อน โดยอาจถูกเรียกผ่านชื่อคลาสเอง (ไม่จำเป็นต้องสร้างอ็อบเจกต์)



สร้าง
(create)

```
House h1 = new House();  
House h2 = new House();
```



การเรียกใช้งานคุณลักษณะหรือเมธอดของคลาส

เมธอดโดยทั่วไปจะไม่มีข้อกำหนด modifier เป็นแบบ static แต่เมธอดที่มี modifier เป็นแบบ static จะสามารถถูกเรียกใช้งาน โดยใช้ชื่อคลาสได้เลยไม่จำเป็นต้องสร้างออบเจกต์ของคลาสนั้นขึ้นมาก่อน ซึ่งมีรูปแบบดังนี้

```
ClassName.methodName();  
ClassName.Attribute;
```

ตัวอย่างเช่น คุณลักษณะและเมธอดในคลาส Math ที่เป็นแบบ static

```
Math.sqrt(4);  
Math.PI;
```

เมธอดแบบ static จะไม่สามารถเรียกใช้เมธอดหรือตัวแปรของออบเจกต์ได้โดยตรง

Field Summary

Fields

Modifier and Type	Field and Description
static double	E The double value that is closer than any other to e , the base of the natural logarithms.
static double	PI The double value that is closer than any other to π , the ratio of the circumference of a circle to its diameter.

Method Summary

All Methods

Static Methods

Concrete Methods

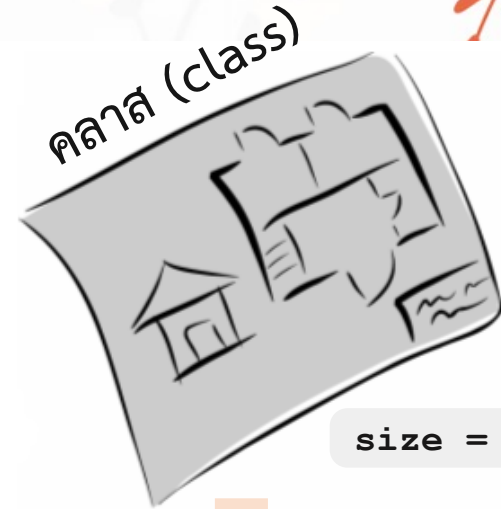
Modifier and Type	Method and Description
static double	abs (double a) Returns the absolute value of a double value.
static float	abs (float a) Returns the absolute value of a float value.
static int	abs (int a) Returns the absolute value of an int value.
static long	abs (long a) Returns the absolute value of a long value.
static double	acos (double a) Returns the arc cosine of a value; the returned angle is in the range 0.0 through π .
static int	addExact (int x, int y) Returns the sum of its arguments, throwing an exception if the result overflows an int.
static long	addExact (long x, long y) Returns the sum of its arguments, throwing an exception if the result overflows a long.
static double	asin (double a) Returns the arc sine of a value; the returned angle is in the range $-\pi/2$ through $\pi/2$.
static double	atan (double a) Returns the arc tangent of a value; the returned angle is in the range $-\pi/2$ through $\pi/2$.
static double	atan2 (double y, double x) Returns the angle θ from the conversion of rectangular coordinates (x, y) to polar coordinates (r, θ).

ตัวอย่าง

```
public class House{  
    public static int size;  
    public int house_id;  
    public void printDetail(){  
        System.out.println("House "+house_id+" "+size);  
    }  
    public static void main(String[] args) {  
        House h1 = new House();  
        h1.size = 32;  
        h1.house_id = 2102;  
        h1.printDetail();  
  
        House h2 = new House();  
        h2.size = 63;  
        h2.house_id = 282;  
  
        h1.printDetail();  
        h2.printDetail();  
    }  
}
```

Output

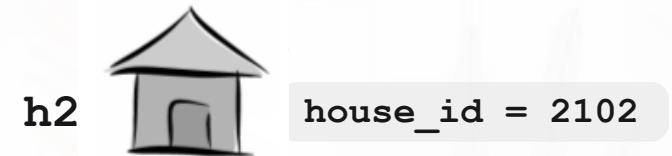
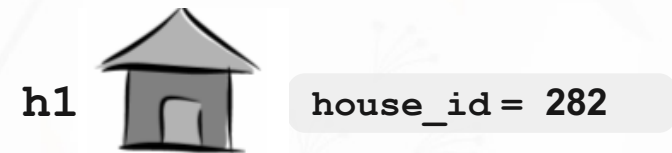
```
House2102 32  
House2102 63  
House282 63
```



size = 32 or 63

สร้าง
(create)

```
House h1 = new House();  
House h2 = new House();
```



วัตถุ (object)

ตัวอย่าง static attribute

```
public class Animal{
    public static int size ;
    public void create(){ size++; }
    public void printDetail(){ System.out.println("number "+ size ); }
}

public class Main{
    public static void main(String[] args) {
        Animal h1 = new Animal();
        h1.create();    h1.printDetail();
        Animal h2 = new Animal();
        h2.create();    h1.printDetail();    h2.printDetail();
        Animal h3 = new Animal();
        h3.create();    h1.printDetail();    h2.printDetail();
        h3.printDetail();
    }
}
```

Output

```
number 1
number 2
number 2
number 3
number 3
number 3
```


ตัวอย่างการใช้งานเมธอดของคลาส

```
public class Main{  
    public static void main(String[] args) {  
        int m = Math.round(12.5);  
        int n = String.valueOf(1234);  
    }  
}
```

↳ แปลง double เป็น int

ตัวอย่างการสร้างแอตทริบิวต์ของคลาส

```
public class Dog {  
    public static final int MAX_LEG = 4;  
    ....  
}
```

↳ ส่วนมาก attribute ที่ static จะเป็น final (แก้ไขไม่ได้)

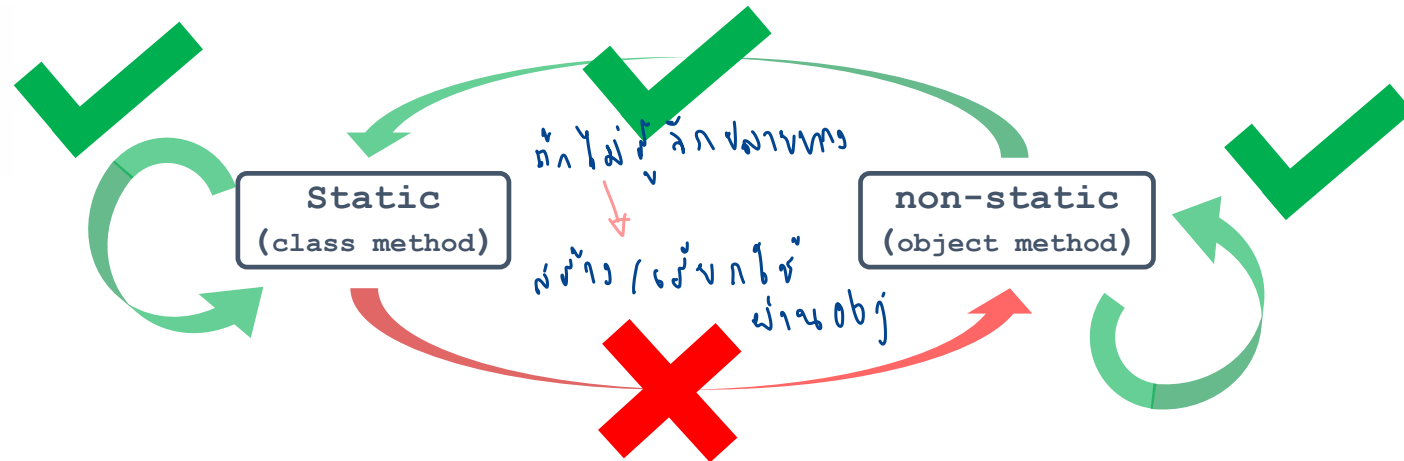
ข้อกำหนดคุณลักษณะหรือเมธอดของคลาส

ประเด็นสำคัญสำหรับคุณลักษณะหรือเมธอดของคลาส (Class attribute and method)

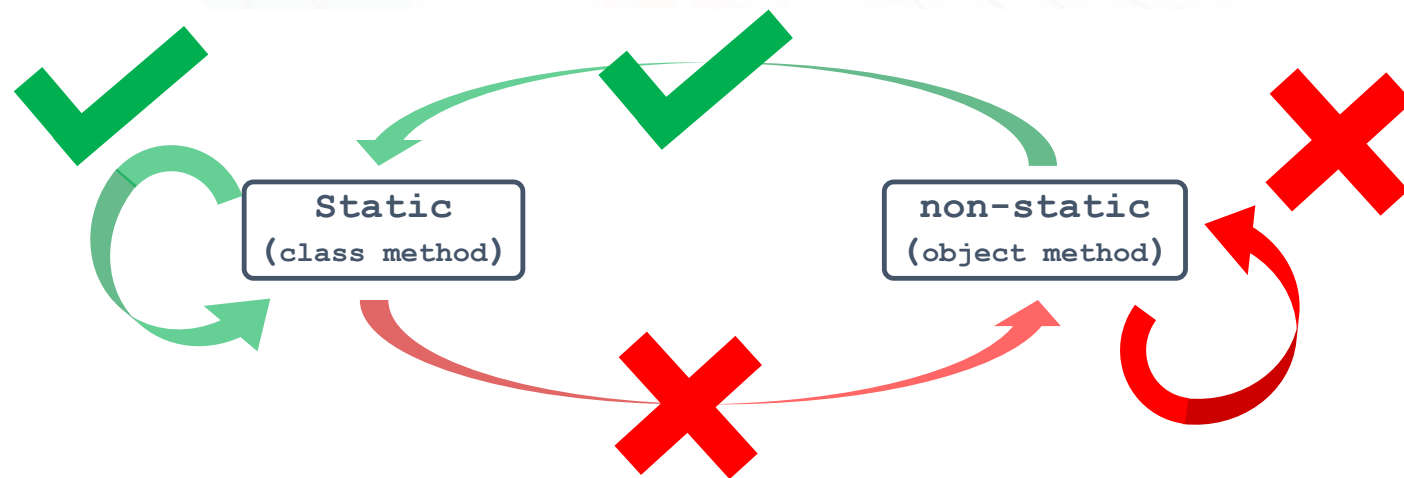
- **แอททริบิวต์ของคลาส** ส่วนมากจะใช้สำหรับเป็นค่าคงหรือค่าที่ให้ทุกคลาสเห็นและสามารถใช้งานได้ (ส่วนใหญ่จะประกาศเป็น `public static final` type ATTRIBUTE_NAME = X ;)
- **เมธอดของคลาส** ไม่สามารถ ~~overridden~~ ได้ แต่สามารถ ~~overloaded~~ ✓ เพื่อจะนำไปใช้แก้ปัญหาสำหรับ static binding ณ ขณะ compiler ประมวลผล
- **เมธอดของคลาส** สามารถเรียกใช้งานได้โดยตรงผ่านชื่อคลาส โดยไม่จำเป็นต้องสร้างวัตถุขึ้นมาก่อน
- **เมธอดของคลาส** ได้ออกแบบให้อยู่บนพื้นฐานที่ว่าแชร์ทรัพยากรร่วมกันระหว่างวัตถุที่สร้างมาจากคลาสเดียวกัน

ความสัมพันธ์ระหว่าง static และ ไม่ static

การเรียกใช้เมธอดภายในคลาสเดียวกัน



การเรียกใช้เมธอดระหว่างคลาส



ซึ่งความสัมพันธ์ดังกล่าวสอดคล้องกันทั้งในกรณี attribute และ method

ตัวอย่าง

```
public class Main{
    public static void main(String[] args) {
        House h = new House();
        h.size = 32; # เติบโตจากคลาส/obj ไม่ขึ้นกับ
        h.house_id = 2102;
                                ↓
                                ตัวแปร static

        h.printSize();
        h.printHouseID();
        h.printDetail1();
        h.printDetail2();
        h.printHi();

        House.printSize();
        House.printHi();

        //House.printHouseID();
        //House.printDetail1(); # ไม่ static เติบโต
        //House.printDetail2(); # ขึ้นกับ class ไม่ได้
    }
}
```

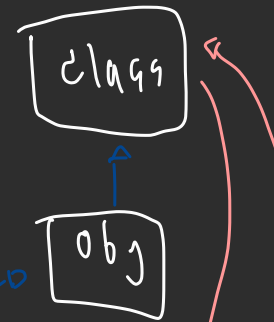
```
public class House{
    public static int size ;
    public int house_id ;

    public static void printHi() {
        System.out.println(" Hi ");
        printSize();
    } public static void printSize() {
        System.out.println("Size "+ size);
    } public void printHouseID() {
        System.out.println("House "+ house_id);
    } public void printDetail1() {
        printSize();
    } public void printDetail2() {
        printHouseID();
    }
}
```

ตัวอย่างการใช้งานเมธอดของคลาส

```
public class Main {  
    public static void main(String[] args) {  
        ① GameCountThree p1 = new GameCountThree();  
        ② p1.count();  
        ③ // GameCountThree.count();  
        ④ // new GameCountThree().count();  
    }  
}
```

```
public class GameCountThree {  
    public static int countNumber; default 0  
    public static void count(){  
        countNumber++; 1  
        System.out.println(countNumber);  
        //System.out.println(this.countNumber); // Error  
        System.out.println(GameCountThree.countNumber);  
    }  
}
```



Static_INITIALIZER static box

Static_INITIALIZER คือบล็อกในคลาสที่อยู่นอกเมธอด และมีคีย์เวิร์ด static เพื่อกำหนดให้เป็นบล็อกแบบ static รูปแบบของ Static_INITIALIZER

```
static {  
    ...          # งาน setup , config  
}
```

คำสั่งในบล็อกแบบ static จะถูกเรียกใช้เพียงครั้งเดียวเมื่อ JVM โหลดคลาสดังกล่าวขึ้นมา ซึ่ง Static_INITIALIZER ใช้ในการดีบั๊ก (debug) โปรแกรม หรือใช้ในกรณีที่ต้องการสร้างอ็อบเจกต์ของคลาสขึ้นโดยอัตโนมัติ นอกจากนี้ เราสามารถที่จะกำหนดบล็อกแบบ static ได้มากกว่าหนึ่งบล็อก โดยการทำงานจะถูกเรียงจากบนลงล่าง

Static_INITIALIZER

```
public class Main {  
    ① [static int x = 5;  
    ②.1 {  
        x += 1; }  
    ③ {  
        public static void main(String args[]) {  
            System.out.println("x = " + x);  
        }  
    ②.2 {  
        x /= 2;  
        6 / 2 = 3  
    }  
}
```

Diagram illustrating the execution flow of the static initializer:

- ①: Declaration of static variable `x` with initial value 5.
- ②.1: First static initializer block where `x` is incremented by 1 (5 + 1 = 6).
- ③: `main` method where `x` is printed.
- ②.2: Second static initializer block where `x` is divided by 2 (6 / 2 = 3).

An arrow indicates the flow from the second static initializer block (②.2) to the `main` method (③), showing that the division occurs before the final print statement.

ผลลัพธ์

x = 3

Static_INITIALIZER

```
public class Main {  
    static int x = 5;  
    static {  
        x += 1;  
    }  
    public static void main(String args[]) {  
        System.out.println("x = "+x);  
        Main m = new Main(); # สร้าง obj ใหม่ ไม่สัวงก็ print ค่าเดิม เพราะเป็นของ class  
        System.out.println("x = "+x);  
    }  
    static {  
        x /= 2;  
    }  
}
```

ผลลัพธ์

x = 3

x = 3